Department of
Computer Science and Engineering

Title: Multi-threaded Programming in Java

Object Oriented Programming Lab
CSE 202

Green University of Bangladesh

1 Objective(s)

After this lecture, the students will be able to:

- Understand and implement java thread
- Apply thread in different use cases

2 Problem analysis

The lab is designed to introduce the students to get the basic idea on the java thread programming. In this lab the basics of java thread, their implementation, and different applications of thread will be taught. To start, we will solve the following problem:

Take two integers M and N as input from the user. Generate all the prime numbers from 1 to M. Divide the input range among N number of threads and generate the numbers in parallel. Show the outputs including which thread generated which number.

3 Theory

3.1 Multitasking

Multitasking is a process of executing multiple tasks simultaneously. We use multitasking to utilize the CPU. Multitasking can be achieved by two ways:

3.1.1 Process-based Multitasking(Multiprocessing)

1. Each process have its own address in memory i.e. each process allocates separate memory area.
2. Process is heavyweight.
3. Cost of communication between the processes is high.
4. Switching from one process to another require some time for saving and loading registers, memory maps, updating lists etc.

3.1.2 Thread-based Multitasking (Multithreading)

1. Threads share the same address space.
2. Thread is lightweight.
3. Cost of communication between the thread is low.

3.2 Lifecycle of a Thread

A thread is a lightweight sub process, a smallest unit of processing. It is a separate path of execution. Threads are independent, if there occurs exception in one thread, it doesn't affect other threads. But threads of a process share a common memory area.

The life cycle of the thread in java is controlled by JVM. The java thread states are as follows:

1. **New** – When we create an instance of Thread class, a thread is in a new state.
2. **Runnable** – The thread is in runnable state after invocation of start() method, but the thread scheduler has not selected it to be the running thread.
3. **Running**– The thread is in running state if the thread scheduler has selected it.
4. **Blocked/Non-Runnable** – This is the state when the thread is still alive, but is currently not eligible to run.
5. **Terminated** – A thread can be terminated, which halts its execution immediately at any given time. Once a thread is terminated, it cannot be resumed. A thread is in terminated or dead state when its run() method exits.

© Dept. of Computer Science and Engineering, GUB

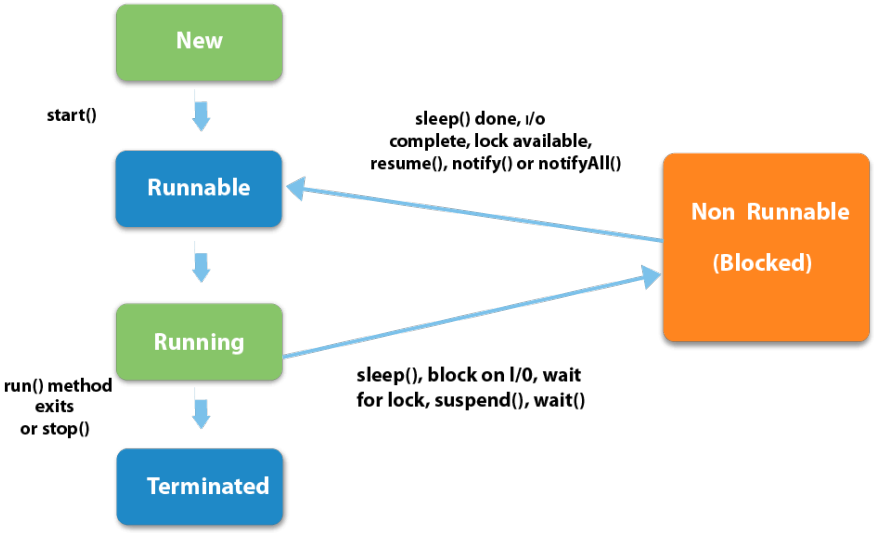

Figure 1: Lifecycle of a Thread

3.3 Creating Thread

There are two ways to create a thread:

1. By extending Thread class
2. By implementing Runnable interface

3.3.1 Creating thread by extending Thread class

Thread class provide constructors and methods to create and perform operations on a thread. Thread class extends Object class and implements Runnable interface. Commonly used constructors of Thread class:

1. Thread()
2. Thread(String name)
3. Thread(Runnable r)
4. Thread(Runnable r, String name)

3.3.2 Creating thread by implementing Runnable interface

The Runnable interface should be implemented by any class whose instances are intended to be executed by a thread. Runnable interface have only one method named run().
public void run(): is used to perform action for a thread.

3.4 Sleeping Thread

Thread.sleep causes the current thread to suspend execution for a specified period. As you know that, at a time, only one thread is executed.If you sleep a thread for the specified time, the thread scheduler picks up another thread and so on. This is an efficient means of making processor time available to the other threads of an application or other applications that might be running on a computer system.

The Thread class provides two methods for sleeping a thread:

1. public static void sleep(long miliseconds)throws InterruptedException
2. public static void sleep(long miliseconds, int nanos)throws InterruptedException

4 Algorithm

Problem Statement: Take two integers M and N as input from the user. Generate all the prime numbers from 1 to M. Divide the input range among N number of threads and generate the numbers in parallel. Show the outputs including which thread generated which number.

Algorithm 1: Using Threads to generate prime numbers

Input: int M, int N

Step 1: Take two integers M and N as input.
Step 2: Create N threads and divide the input range 1 to M into N threads
Step 3: Each thread will generate and print the prime numbers in its defined range.
Step 4: Exit

5 Implementation

```java

public class PrimeTest extends Thread {
    int start;
    int end;
    int id;
    PrimeTest(int id, int start,int end)
    {
        this.start = start;
        this.end = end;
        this.id = id;
    }

   public void run(){
       for(int i=start;i<=end;i++)
       {
           if(checkPrime(i)==true)
           {
               System.out.println("Thread "+this.id+" generated "+i);
               try {
                   sleep(100);
               } catch (InterruptedException ex) {
                   System.out.println(ex.getMessage());
               }
           }
       }
   }

   public boolean checkPrime(int x){
        // Skip 0 and 1 as they are neither prime nor composite
        if (x == 1 || x == 0)
            return false;

        for (int i = 2; i <= x / 2; i++) {
            if (x % i == 0) {
                return false;
            }
        }
        return true;
   }
}
```

```java
import java.util.Scanner;
public class Test{
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.println("Enter M = Maximum Value");
        int M = sc.nextInt();
        System.out.println("Enter N = Number of Threads");
        int N = sc.nextInt();
        int start =0;
        int rangeSize = M/N;
        for(int i =1;i<=N;i++){
            PrimeTest t = new PrimeTest(i,start+1,start + rangeSize);
            start = start + rangeSize;
            t.start();
        }
    }
}
```

6 Input/Output

Input : M = 500
N = 5
Output of the program is given below.

The prime numbers from 1 to 100 are highlighted in green in Figure 2.

Thread 1 generated : 2
Thread 2 generated : 101
Thread 1 generated : 3
Thread 3 generated : 211
.
.
.

1	2	3	4	5	6	7	8	9	10
11	12	13	14	15	16	17	18	19	20
21	22	23	24	25	26	27	28	29	30
31	32	33	34	35	36	37	38	39	40
41	42	43	44	45	46	47	48	49	50
51	52	53	54	55	56	57	58	59	60
61	62	63	64	65	66	67	68	69	70
71	72	73	74	75	76	77	78	79	80
81	82	83	84	85	86	87	88	89	90
91	92	93	94	95	96	97	98	99	100

Figure 2: List of prime numbers

© Dept. of Computer Science and Engineering, GUB

7 Discussion & Conclusion

From this experiment, we use the concept of finding prime numbers to learn about how to write a program using thread in java. Try to answer the following questions :

1. Why do we pass start + 1 value in line 13 when initializing a thread in main function.
2. Why do we call the sleep method in line 21 of PrimeTest class.
3. We called the start function of PrimeTest object in line 15 of Test class. But there is no start method in PrimeTest class. Why is there no error ?
4. Why do we return false in line 36 of PrimeTest class.

8 Lab Task (Please implement yourself and show the output to the instructor)

Search for a number N inside a randomly generated array of size 200 using 5 threads.

8.1 Problem analysis

Create an integer array, arr of size 200 and generate 200 random numbers for that array. Write a program where you need to search a number in that given array, arr. Distribute the given array into five threads and perform the search in parallel. Suppose one thread searches for the input number from the index 0 to 40. Another search can be performed from the index 41-80 in another thread.

Take an input N. Now search for N in the array using 5 threads. The algorithm is given below in Algorithm 2.

9 Algorithm

Algorithm 2: Search using thread

Input: None

```
Create an integer array, arr of size 200
Generate 200 random int numbers for that array
Create 5 threads
Take a number, N as input
Thread t_1 will search for N in the arr[0] to arr[39]
Similarly, Thread t_2 to t_5 will search for N in the other indices.
If thread 1 finds N in the array, print "Found N in Thread 1". Exit
```

10 Lab Exercise (Submit as a report)

Complete the lab task and submit it as lab report.

11 Policy

Copying from internet, classmate, seniors, or from any other source is strongly prohibited. 100% marks will be *deducted* if any such copying is detected.

12 References

https://www.edureka.co/blog/java-thread/

13 Appendix

If required, course instructor might provide some additional materials.

© Dept. of Computer Science and Engineering, GUB